# PROYECTO DE PROGRAMACIÓN 1ER SEMESTRE

Wondershare PDFelement

**Nombre**: Dylan Ramsés Cabrera Morales
**Grupo**: 122

Este proyecto consiste en un sistema de recuperación de información que tiene como objetivo realizar una búsqueda en un conjunto de archivos. La idea central de este es facilitar la búsqueda del usuario y hacerla lo más óptima posible. La interfaz gráfica ha sido programada en su mayoría con HTML y CSS, mientras que el código de la búsqueda fue implementado en C#.
Para ejecutar el proyecto debe leer Moogle.md

Las clases utilizadas para la lógica del proyecto son:
**ProcesarDocumentos, Busqueda, Consulta, Moogle, SearchItem, SearchResult y Matriz.**

Al ejecutarse el proyecto inmediatamente se procede a llamar a la clase **ProcesarDocumentos**. Dicha clase es la encargada de almacenar todos los archivos de texto que se encuentran ubicados en la carpeta Content en una Lista que contiene una Lista de las palabras de cada documento(Lista de Listas).
Este proceso lo realiza el método **CargarDocumentos** de la clase **ProcesarDocumentos**. En este se ejecutan las siguientes funciones:

.LeerTextos: Devuelve una lista de documentos que contiene un listado de cada palabra que aparece en un documento. Para lograr esto , se invocan los métodos **ObtenerDocumentos** y **Tokenizar.**
**ObtenerDocumentos** devuelve las rutas de todos los documentos de la carpeta Content, mientras que Tokenizar se encarga de evitar la aparición de signos de puntuación(esto lo realiza mediante la función ModificarTexto) y luego separa las palabras y las almacena en una lista de palabras

```
public static List<List<string>>LeerTextos()//Devuelve una lista de los documentos que contiene y a su vez una lista de las palabras
{
    List<List<string>>documentos = new();
    string[]nombredocumentos = ObtenerDocumentos();
    char[] separadores = {' ','=','`',';','\','\t','.',',',':','-','_','/','+','%','?','[',']','(',')','{','}','|','<','>','!','#','&','@'};
    for(int i = 0;i<nombredocumentos.Length;i++)
    {
      documentos.Add(Tokenizar(File.ReadAllText(nombredocumentos[i]),separadores));
    }
    return documentos;
}
```

Wondershare PDFelement

.CalcularTFIDF: Su objetivo es calcular el TFIDF de todos los documentos (Para más información sobre este algoritmo haga click aquí). Al finalizar su ejecución tendremos los valores del TF almacenados en una lista de diccionarios y el IDF en un diccionario independiente

```
public static void CalcularTFIDF(Dictionary<string,double>IDF,List<Dictionary<string,double>>Lista_TF,List<List<string>>documentos)
 {
   for(int i = 0;i<documentos.Count;i++)
   {
      Dictionary<string,double>TF = new();
      for(int j = 0;j<documentos[i].Count;j++)
      {
         if(!TF.ContainsKey(documentos[i][j]))
         {
            TF.Add(documentos[i][j],1);
         }
         else
         {
            TF[documentos[i][j]]++;
         }
         if(!IDF.ContainsKey(documentos[i][j]))
         {
            IDF.Add(documentos[i][j],1);
         }
         else if(TF[documentos[i][j]]==1)
         {
            IDF[documentos[i][j]]++;
         }
      }
      Lista_TF.Add(TF);
   }
```

.TFIDF: Devuelve una lista de diccionarios con el valor del TFIDF de cada documento

Wondershare PDFelement

```
public static List<Dictionary<string,double>>TFIDF(Dictionary<string,double>idf,List<Dictionary<string,double>>Lista_tf)
{
 List<Dictionary<string,double>>TFIDF = new();
 foreach(Dictionary<string,double>tfdic in Lista_tf)
 {
  foreach(KeyValuePair<string,double>elemento in tfdic)
  {
     tfdic[elemento.Key] *= idf[elemento.Key];
  }
  TFIDF.Add(tfdic);
 }
 return TFIDF;
}
```

.Magnitud_Documentos: A partir del empleo del modelo vectorial, este método calcula la magnitud de un documento y lo devuelve como un array de valores de tipo double. Este es un cálculo que solo se ejecuta una vez ya que no depende de la búsqueda(Para más información haga click aquí)

Wondershare PDFelement

```
1   public static double[]Magnitud_Documentos(List<Dictionary<string,double>>tfidfs_doc)
2   {
3     double[]magnitud_documentos = new double[tfidfs_doc.Count];
4     for(int i = 0;i<magnitud_documentos.Length;i++)
5     {
6        double suma = 0;
7        foreach(KeyValuePair<string,double>elemento in tfidfs_doc[i])
8        {
9         suma += elemento.Value*elemento.Value;
10       }
11       magnitud_documentos[i] = Math.Sqrt(suma);
12    }
13    return magnitud_documentos;
14  }
15 }
```

Con esta información almacenada, se proceden a ejecutar las instrucciones de la clase Moogle. Una vez se introduce la búsqueda la clase Moogle llama al método Tokenizar de la clase ProcesarDocumentos, para separar las palabras de la query y almacenarlas en una lista de palabras. Luego se ejecutan una serie de funciones de la clase Consulta:

.TFIDF_Query: Se encarga de calcular el TFIDF de la query

Wondershare PDFelement

```
1  public static Dictionary<string,double>TFIDF_Query(Dictionary<string,double>idf,List<string>query)
2  {
3      Dictionary<string,double>tfidf_query = new();
4      foreach(string palabra in query)
5      {
6          if(!tfidf_query.ContainsKey(palabra))
7          {
8              tfidf_query.Add(palabra,1);
9          }
10         else
11         {
12             tfidf_query[palabra]++;
13         }
14     }
15     foreach(KeyValuePair<string,double>palabra in tfidf_query)
16     {
17         if(idf.ContainsKey(palabra.Key))
18         {
19             tfidf_query[palabra.Key] *= idf[palabra.Key];
20         }
21         else
22         {
23             tfidf_query[palabra.Key] = 0;
24         }
25     }
26     return tfidf_query;
27 }
```

Wondershare PDFelement

.Magnitud_Query: Calcula la magnitud de la query

```
public static double Magnitud_Query(Dictionary<string,double>tfidf_query)
{
    double suma_query = 0;
    foreach(KeyValuePair<string,double>elemento in tfidf_query)
    {
        suma_query += elemento.Value*elemento.Value;
    }
    return Math.Sqrt(suma_query);
}
```

.SimilitudCosenos: Emplea el modelo vectorial para calcular la similitud de cosenos de los documentos y lo devuelve como un array de double

```
public static double[] SimilitudCosenos(Dictionary<string,double>query_tfidf,List<Dictionary<string,double>>lista_tfidf,double magnitud_query,double[]magnitud_documentos)
    {
        double[] suma = new double[lista_tfidf.Count];
        for(int i = 0;i<lista_tfidf.Count;i++)
        {
            foreach(KeyValuePair<string,double>elemento in query_tfidf)
            {
                if(lista_tfidf[i].ContainsKey(elemento.Key))
                {
                    suma[i] += elemento.Value*lista_tfidf[i][elemento.Key];
                }
            }
        }
        double[] similitudcosenos = new double[lista_tfidf.Count];
        for(int i = 0;i<similitudcosenos.Length;i++)
        {
            double magnitudes = magnitud_query*magnitud_documentos[i];
            if(magnitudes!=0)
            {
                similitudcosenos[i] = (suma[i]/magnitudes);
            }
            else
            {
                similitudcosenos[i] = 0;
            }
        }
        return similitudcosenos;
    }
```

Wondershare PDFelement

En este punto ya tendremos los documentos más relevantes con respecto a la búsqueda, donde entra en juego la clase Busqueda para obtener un fragmento del documento que contiene parte de la búsqueda. Desde la clase principal Moogle se llaman los siguientes métodos de Busqueda:

.PosiciondelosDocumentosImportantes: Es el método que devuelve la posición que ocupan en el array de similitud de cosenos los documentos más importantes. Si no se encuentra ningún documento similar retorna -1

A partir de aquí se divide en dos variantes: si se encuentran los resultados y si no se encuentran.

Si son encontrados:

.DocumentosmasImportantes: Devuelve la ruta de los documentos más importantes ordenados

Wondershare PDFelement

```
public static string[] Documentosmasimportantes(double[]cosenos,int[]posiciondedocumentos)
  {
      string[]documentosmasimportantes = new string[posiciondedocumentos.Length];
      for(int i = 0;i<documentosmasimportantes.Length;i++)
      {
       documentosmasimportantes[i] = ProcesarDocumentos.ObtenerDocumentos()[posiciondedocumentos[i]];
      }
      return documentosmasimportantes;
  }
```

.ObtenerSnippet: Devuelve un fragmento del texto original que contenga parte del query

Si no son encontrados:
Se aplica un algoritmo de distancia entre palabras para encontrar aquellas que más se parecen a la query. Se llama al método Sugerencia de la clase Consulta que realiza esta función mediante el método Similar. Esto permite al usuario la opción de realizar una nueva búsqueda que si tendrá resultados.

La clase Matriz nunca se utiliza. Fue implementada para la evaluación de Álgebra.

Wondershare PDFelement